

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного програмування

Звіт

Про виконання лабораторної роботи №1
«Інтерполяція кубічними сплайнами.»

Виконав:

Студент групи ФЕІ-12

Студент Чипіжук З. М.

Перевірив:

ас. Патинко А. М.

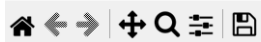
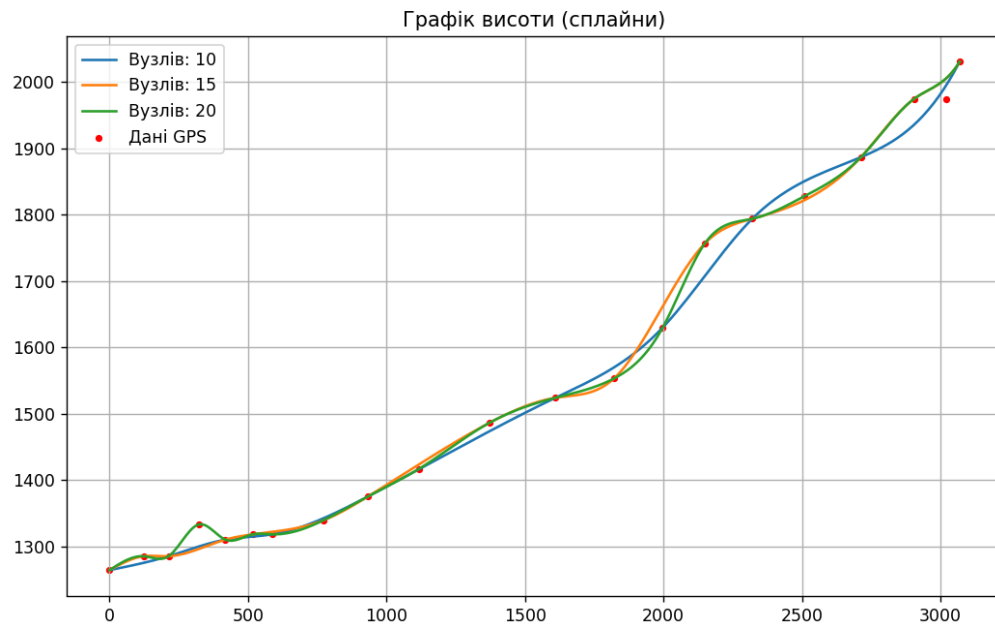
Львів 2026

Мета: вивчити метод інтерполяції кубічними сплайнами.

Хід роботи:

1. **Запит до API:** Виконано запит до сервісу Open-Elevation для отримання географічних даних (широта, довгота) та значень висоти (elevation) для точок маршруту.
2. **Табуляція даних:** Результати API-запиту опрацьовано та представлено у вигляді таблиці з індексами, координатами та значеннями висоти
3. **Розрахунок відстаней:** Використано формулу гаверсинусів для обчислення відстаней між сусідніми точками. Сформовано масив кумулятивної відстані (distances) для побудови графіка
4. Ми наближаємо профіль маршруту кубічними поліномами, що з'єднуються у вузлах. Для забезпечення гладкості кривої в точках з'єднання виконуються умови неперервності значень та їхніх похідних, а коефіцієнти обчислюються методом прогонки.
5. **Візуалізація:** Побудовано графіки інтерполяції для 10, 15 та 20 вузлів для оцінки точності наближення.
6. **Додаткове завдання:** Додатково було проведено розширений аналіз маршруту. Обчислено загальну довжину шляху, сумарний набір висоти та загальний спуск, що дозволило оцінити фізичну складність підйому. Також виконано розрахунок градієнта (крутизни) маршруту через визначення швидкості зміни висоти на кожній ділянці, виявлено ділянки з крутизною понад 15%. Окремо розраховано механічну роботу, необхідну для підйому людини вагою 80 кг, на основі зміни потенціальної енергії залежно від сумарного набору висоти.

Результат роботи:



Табуляція (відстань, висота):

i	Distance (m)	Elevation (m)
0	0.00	1264.00
1	123.85	1285.00
2	213.45	1285.00
3	323.50	1333.00
4	418.87	1310.00
5	517.41	1318.00
6	587.78	1318.00
7	771.72	1339.00
8	932.77	1375.00
9	1118.27	1417.00
10	1370.81	1486.00
11	1610.05	1524.00
12	1821.16	1553.00
13	1997.39	1630.00
14	2149.88	1757.00
15	2320.63	1794.00
16	2508.86	1828.00
17	2713.03	1887.00
18	2904.98	1975.00
19	3020.67	1975.00
20	3069.34	2031.00

Коефіцієнти сплайна (перші 3 інтервали):

i	a	b	c	d
0	1264.00	0.2703	0.000000	-0.00000657
1	1285.00	-0.0319	-0.002439	0.00003119
2	1285.00	0.2823	0.005945	-0.00004132

--- ХАРАКТЕРИСТИКИ МАРШРУТУ ---

Загальна довжина (м): 3069.34

Сумарний набір висоти (м): 790.0

Сумарний спуск (м): 23.0

Механічна робота (кДж): 619.99

Process finished with exit code 0

Код програми:

```
import requests
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
url = "https://api.open-elevation.com/api/v1/lookup?locations=48.164214,24.536044|48.164983,24.534836|48.165605,24.534068|48.166228,24.532915|48.166777,24.531927|48.167326,24.530884|48.167011,24.530061|48.166053,24.528039|48.166655,24.526064|48.166497,24.523574|48.166128,24.520214|48.165416,24.517170|48.164546,24.514640|48.163412,24.512980|48.162331,24.511715|48.162015,24.509462|48.162147,24.506932|48.161751,24.504244|48.161197,24.501793|48.160580,24.500537|48.160250,24.500106"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
results = data["results"]
```

```
n = len(results)
```

```
print("Кількість вузлів:", n)
```

```
def haversine(lat1, lon1, lat2, lon2):
```

```
    R = 6371000
```

```
    p1 = np.radians(lat1)
```

```
    p2 = np.radians(lat2)
```

```
    d_lat = np.radians(lat2 - lat1)
```

```
    d_lon = np.radians(lon2 - lon1)
```

```
    a = np.sin(d_lat / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(d_lon / 2) ** 2
```

```
    return 2 * R * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
```

```
    distances = [0]
```

```
    elevations = []
```

```
    for i in range(n):
```

```

        elevations.append(results[i]["elevation"])

for i in range(1, n):

    d = haversine(results[i - 1]["latitude"], results[i - 1]["longitude"], results[i]
["latitude"], results[i]["longitude"])

    distances.append(distances[i - 1] + d)

print("\nТабуляція (відстань, висота):")

print(" i | Distance (m) | Elevation (m)")

for i in range(n):

    print(f"{i:2d} | {distances[i]:10.2f} | {elevations[i]:8.2f}")

def my_spline(x, y):

    num = len(x) - 1

    h = []

    for i in range(num):

        h.append(x[i + 1] - x[i])

    a1 = np.zeros(num + 1)
    be = np.zeros(num + 1)
    ga = np.zeros(num + 1)
    de = np.zeros(num + 1)

    be[0] = 1
    de[0] = 0

    for i in range(1, num):

        a1[i] = h[i - 1]
        be[i] = 2 * (h[i - 1] + h[i])
        ga[i] = h[i]
        de[i] = 3 * ((y[i + 1] - y[i]) / h[i] - (y[i] - y[i - 1]) / h[i
- 1])

    a1[num] = h[num - 1]
    be[num] = 2 * h[num - 1]
    de[num] = 3 * ((y[num] - y[num - 1]) / h[num - 1])

```

```

A = np.zeros(num + 1)
B = np.zeros(num + 1)
A[0] = -ga[0] / be[0]
B[0] = de[0] / be[0]
for i in range(1, num + 1):
    den = al[i] * A[i - 1] + be[i]
    if i < num:
        A[i] = -ga[i] / den
    B[i] = (de[i] - al[i] * B[i - 1]) / den

c = np.zeros(num + 1)
c[num] = B[num]
for i in range(num - 1, -1, -1):
    c[i] = A[i] * c[i + 1] + B[i]

a_coeffs = y[:-1]
b_coeffs = np.zeros(num)
d_coeffs = np.zeros(num)
for i in range(num):
    d_coeffs[i] = (c[i + 1] - c[i]) / (3 * h[i])
    b_coeffs[i] = (y[i + 1] - y[i]) / h[i] - (h[i] / 3) * (c[i + 1]
+ 2 * c[i])

return a_coeffs, b_coeffs, c[:-1], d_coeffs

counts = [10, 15, 20]

plt.figure(figsize=(10, 6))

for c_nodes in counts:

    idx = np.linspace(0, n - 1, c_nodes).astype(int)

    xs = np.array(distances)[idx]

    ys = np.array(elevations)[idx]

a, b, c, d = my_spline(xs, ys)

x_fine = []
y_fine = []
for i in range(len(xs) - 1):

```

```

    step = np.linspace(xs[i], xs[i + 1], 20)
    for val in step:
        dx = val - xs[i]
        y_val = a[i] + b[i] * dx + c[i] * (dx ** 2) + d[i] * (dx **
3)
        x_fine.append(val)
        y_fine.append(y_val)

plt.plot(x_fine, y_fine, label=f"Вузлів: {c_nodes}")

if c_nodes == 20:
    print("\nКоефіцієнти сплайна (перші 3 інтервали):")
    print(" i |    a    |    b    |    c    |    d")
    for i in range(3):
        print(f"{i:2d} | {a[i]:7.2f} | {b[i]:7.4f} | {c[i]:7.6f} |
{d[i]:7.8f}")

plt.scatter(distances, elevations, color='red', s=10, label="Дані GPS")

plt.legend()

plt.grid()

plt.title("Графік висоти (сплайни)")

plt.show()

ascent = 0

for i in range(1, n):

    diff = elevations[i] - elevations[i - 1]

    if diff > 0:

        ascent += diff

descent = 0

for i in range(1, n):

    diff = elevations[i - 1] - elevations[i]

    if diff > 0:

```



```
        descent += diff

print("\n--- ХАРАКТЕРИСТИКИ МАРШРУТУ ---")

print("Загальна довжина (м):", round(distances[-1], 2))

print("Сумарний набір висоти (м):", round(ascent, 2))

print("Сумарний спуск (м):", round(descent, 2))

work = 80 * 9.81 * ascent

print("Механічна робота (кДж):", round(work / 1000, 2))

f = open("results.txt", "w", encoding="utf-8")

f.write("РЕЗУЛЬТАТИ\n")

f.write(f"Довжина: {distances[-1]}\n")

f.write(f"Набір: {ascent}\n")

f.write(f"Спуск: {descent}\n")

f.close()
```

Репозиторій:

https://github.com/Zakhar799/numerical_methods_2026/tree/master

Висновок:

У ході роботи було вивчено метод інтерполяції кубічними сплайнами як математичну модель пружного стержня.

Реалізовано алгоритм побудови сплайна з використанням методу прогонки для розв'язання трьохдіагональної системи рівнянь, що забезпечило гладкість профілю висоти на всьому маршруті. Чисельні експерименти показали, що кількість вузлів безпосередньо впливає на точність апроксимації рельєфу. Запропонований підхід дозволяє ефективно обробляти GPS-дані для прогнозування характеристик маршруту.